

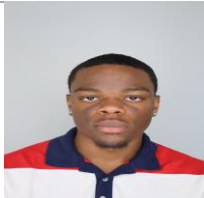
[SP-27-Red Spotify]

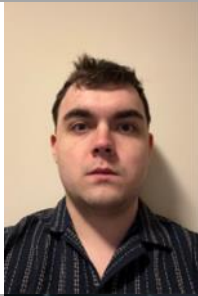
CS 4850, Section 02, Spring Semester 2026

April 12

Sharon Perry

Software Test Plan/Report

Team Role	Author	Bio Pic
Developer	Davon Appolon	
Tester	Isaiah Langford	

Documenter	Andrew Henshaw		
Developer	Rowan Crooks-Taylor		

1. Test Objectives	2
2. Scope of Testing.....	2
2.1. In Scope	2
2.2. Out of Scope.....	2
3. Test Cases	2
4. Test Procedures.....	2
4.1. App Launch	2
4.2. Spotify Login Redirect	2
4.3. Successful Authentication	2
4.4. Token Exchange.....	2
4.5. Fetch User Profile	2
4.6. Fetch Playlists.....	2
4.7. Token Expiration Handling.....	2
4.8. Invalid Token Handling.....	2
4.9. Backend Endpoint Test.....	2
4.10. UI Navigation	2
5. Test Environment	2
5.1. Hardware Environment.....	2
5.2. Software Environment	2
5.3. External Services.....	2
5.4. Network Environment.....	2
5.5. Test Data.....	2
5.6. Environment Configuration	2
5.7. Limitations	2
6. Software Test Report	2
6.1. Security.....	2

6.2. Stress & Volume Testing (S&V)	2
6.3. Connectivity Testing (CT)	2
6.4. Disaster Recovery/Back Up	2
6.5. Unit Testing	2
6.6. Integration Testing	2
7. Software Test Report	2
7.1. Test level responsibility	2
7.2. Test Type & Approach	2
7.3. Build strategy	2
7.4. Test Execution Schedule	2
7.5. Facility, data, and resource provision plan	2
7.5.1. Test environment	2
7.5.2. Access to other applications	2
7.5.3 Testing Requirements	2
7.5.4. Data Requirements	2
7.5.5. Resources & Skills	2
7.6. Testing Tools	2
7.7. Testing Handover Procedure	2
7.8. Testing Metrics	2
8. Test Environment Plan	2
8. Test Environment Man	2
8.1. Test Environment Details	2
8.1.1. Testers	2
8.1.2. Hardware and Firmware	2
8.1.3. Software	2
8.1.4. Interfaces	2
8.1.5. Other Materials	2

8.2. Establishing Environment.....	2
8.3. Environment Control.....	2
8.4. Environment Roles and Responsibilities	2
9. Assumptions and Dependencies.....	2
9.1. Assumptions.....	2
9.2. Dependencies.....	2
10. Entry and Exit Criteria	2
11. Administrative Plan.....	2
11.1. Approvals	2
11.2. Test Milestones and Schedule	2
11.3. Training	2
11.4. Defect Management.....	2
12. Definitions	2
13. References	2

1. Test Objectives

The objective of this test plan is to make sure that our Red Spotify applications functions correctly, securely, and reliably in its core features.

The testing focuses on verifying:

- *Successful user authentication using Spotify OAuth 2.0*
- *Proper communication between fronted, backend, and Spotify API*
- *Accurate retrieval and display of user data (profile, playlists, activity)*
- *Stability of token handline (access + refresh tokens)*
- *Error handling for failed API requests or invalid tokens*
- *Overall usability and performance of the application*

2. Scope of Testing

2.1. In Scope

The following components will be tested:

- *Authentication System*
 - *Login with Spotify*
 - *Authorization code flow*
 - *Token exchange (backend)*
 - *Token refresh*
- *API Integration*
 - *Fetching user profile*
 - *Fetching playlists*
 - *Handling API errors*
- *Frontend UI*
 - *Navigation between screens*
 - *Display of user data*
 - *Error messages*
- *Backend Service*
 - *Token exchange endpoint (/auth/token)*
 - *Secure handling of credentials*
 - *Communication with Spotify API*

2.2. Out of Scope

- *Spotify internal systems*
 - *Spotify subscription handling*
 - *Offline functionality*
 - *Large-scale load/performance testing*
-

3. Test Cases

Set of test cases used:

- *Test Case 1: App Launch*
 - *Description: Verify the app launches correctly*
 - *Expected Result: Login button and navigation tabs appear*
- *Test Case 2: Spotify Login Redirect*
 - *Description: User clicks "Login with Spotify"*
 - *Expected Result: User is redirected to Spotify login page*
- *Test Case 3: Successful Authentication*
 - *Description: User logs in successfully*
 - *Expected Result: User is redirected back to app with no errors*
- *Test Case 4: Token Exchange*
 - *Description: Frontend sends auth code to backend*
 - *Expected Result: Backend returns access + refresh tokens*
- *Test Case 5: Fetch User Profile*
 - *Description: App requests user profile data*
 - *Expected Result: Profile data is displayed correctly*
- *Test Case 6: Fetch Playlists*
 - *Description: App requests playlists*
 - *Expected Result: playlist data is displayed*
- *Test Case 7: Token Expiration Handling*
 - *Description: Access token expires*
 - *Expected Result: Refresh token generates new access token automatically*
- *Test Case 8: Invalid Token Handling*
 - *Description: Use an invalid/expired token*
 - *Expected Result: Error handled gracefully (no crash, message shown)*
- *Test Case 9: Backend Endpoint Test*
 - *Description: Call /auth/token API*
 - *Expected Result: Proper token response or error handling*
- *Test Case 10: UI Navigation*
 - *Description: Navigate between screens*
 - *Expected Result: Smooth transitions, and no crashes*

4. Test Procedures

4.1. App Launch

- Step 1: App opens
- Step 2: App opens on a mobile device using the Expo Go app
- Step 3: initial screen pops up
- Expected result: Login button and navigation tabs appear

4.2. Spotify Login Redirect

- Step 1: User clicks Login button
- Step 2: Spotify redirect tab pops up
- Expected Result: User is redirected to the Spotify login page

4.3. Successful Authentication

- Step 1: User logs in to Spotify in the redirected tab
- Step 2: User accepts Spotify permissions
- Expected Result: User is logged into Spotify and is redirected back to the app with no errors

4.4. Token Exchange

- Step 1: User accepts authorization from Spotify
- Step 2: Frontend sends request to /auth/token
- Step 3: Backend responds
- Expected Result: Backend returns access token and refresh token

4.5. Fetch User Profile

- Step 1: User accepts Spotify authorization
- Step 2: User is redirected back to the app and the dashboard is visible
- Expected Result: User should see their Spotify dashboard upon being redirected to the app

4.6. Fetch Playlists

- Step 1: User is on the dashboard screen after login
 - Step 2: User navigates to the library tab
 - Expected Result: Playlist data should be displayed correctly
-

4.7. Token Expiration Handling

- Step 1: User logs in with Spotify
- Step 2: Wait for access token to expire or simulate expiration
- Step 3: Trigger API request
- Expected Result: Refresh token generates a new access token automatically

4.8. Invalid Token Handling

- Step 1: Manually use an invalid or expired token
- Step 2: Attempt to fetch data from Spotify API
- Expected Result: Error is handled and message is displayed

4.9. Backend Endpoint Test

- Step 1: Send request to /auth/token endpoint
- Step 2: Give valid / invalid authorization code
- Expected Result: Proper token response or error handling returned

4.10. UI Navigation

- Step 1: Open application (don't have to login to navigate UI)
- Step 2: Navigate the available screens with no issues
- Expected Result: Navigation goes smooth and there are no cr

5. Test Environment

5.1. Hardware Environment

- Mobile devices:
 - Android 10 or higher
 - IOS 14 or higher
- PC/Laptop
 - Computer capable of running Node.js and Expo

5.2. Software Environment

- Frontend:
 - React Native
 - Expo Go mobile app
- Backend:
 - Node.js
 - Hosted on Render
- Development Tools:
 - Visual Studio Code
 - GitHub for version control

5.3. External Services

- Spotify Web API
 - Authorization Endpoint
 - Token Endpoint
 - API Base URL

5.4. Network Environment

- Stable internet connection required
- HTTPS for all communications
- Testing performed under optimal network conditions

5.5. Test Data

- Spotify User Accounts (our accounts)
- Real user playlist and listening data

5.6. Environment Configuration

- Environment variables configured

- SPOTIFY_CLIENT_ID
- OAuth 2.0 Authorization Code Flow with PKCE enabled
- Redirect URL generated dynamically from Expo Auth session

5.7. Limitations

- Dependent on Spotify API availability
- No offline testing
- Valid Spotify account needed

6. Software Test Report

6.1. Security

We'll validate that user information, authentication, and API calls are all protected through our security testing. This will include ensuring the OAuth authentication system is working properly, securely handling access tokens, and validating all API requests for the Spotify Web API, among other things. We will also be testing for things like input validation, proper protection against unauthorized access, and verifying that our system is using HTTPS for all its communications. Security testing will be done by our development team during the build phase and confirmed by our QA/testing team prior to release.

6.2. Stress & Volume Testing (S&V)

We will use Red Spotify for stress and volume testing, in which we will put a lot of users and API calls to it and test its performance, stability, limit, whether it can respond accurately and effectively under the huge pressure of traffic and the way it handles such conditions.

6.3. Connectivity Testing (CT)

We'll perform connectivity tests between our app, back-end, and the Spotify API to simulate various network states and verify that the communication lines remain robust and stable. The testing also covers handling errors appropriately and that the system can recover from connectivity interruption automatically.

6.4. Disaster Recovery/Back Up

Disaster recovery and backup testing requires a simulation to emulate failure and then recover backed-up data from that simulated failure. This testing checks that backups are accurate, how quickly they can be recovered, and ensures data integrity is maintained during a disaster and that applications remain in use throughout an outage or failure.

6.5. Unit Testing

Unit tests verify each unit of a module including an API call, an authentication method, a data handling algorithm, and a UI component, and it ensures that each function works as expected, correctly accepting certain inputs and outputs, and correctly handling different edge cases.

6.6. Integration Testing

Integration testing verifies interactions between frontend, backend, and Spotify API. Testing ensures authentication, data flow, and feature functionality work together correctly, confirming seamless communication and no errors across connected system components.

7. Software Test Report

7.1. Test level responsibility

Test Level	External Party	Proj Team	Business
Unit Testing		P	
Integration Testing		P	
Security Testing	P	S	
Connectivity Testing		P	
User Acceptance Testing		S	P
Production Verification Testing		S	P

7.2. Test Type & Approach

Test Type	Objectives
Progression Requirements	Test objectives include verifying the requirements are met, ensuring accuracy, testing error handling, checking data flow, and verifying system to system interactions. The tests need to be done successfully with a few bugs to move on to the next test. In addition, a successful test requires a successful performance test and all the functionality to be done.
Regression testing	It is done to ensure that new functions or upgrades in Red Spotify don't break the app. It has some objectives, namely: To keep music playback, user authentication, data and other app features stable, to catch regression errors, and to maintain app performance overall.
Functional testing	It confirms that all Red Spotify features meet all established requirements. Objectives: To test the functionality of authentication, playlists, recommendations, and accuracy, to test the user's correct interaction, and to test the application's accurate functioning.
Integration testing	This test suite is designed to verify the proper interoperation of Red Spotify components. Its primary goals are to confirm the integrity of frontend and backend interactions, the validity of Spotify API integration and user authentication flows, and the consistency of all data, with a particular emphasis on ensuring that all components linked to the system function in harmony.
Performance testing	Key objectives involve measuring application response time, supporting a concurrent userbase, assessing the REST API's response time, and keeping Red Spotify stable, fast and responsive, under peak load circumstances.
Security testing	To check that our data and services are properly protected from unauthorized access and misuse. The purpose of this test is to confirm that authentication, authorization and API requests are secured. In addition, to check that there is no unauthorized access to the application and that sensitive user information is not mishandled by the application.

7.3. Build strategy

There will be a phased approach to the build, dividing development into distinct functional requirements like authentication, playlist, analytics and recommendations. This enables us to test, iterate and implement in manageable chunks.

7.4. Test Execution Schedule

Microsoft Word will be used to develop the test schedule. Tables will be created to organize the schedule, including columns for the testing type, the function being tested, the level of priority, and a timeline for the testing.

7.5. Facility, data, and resource provision plan

7.5.1. Test environment

The available test systems will comprise development, staging, and production-like systems. These will include Spotify APIs and test data sets. The availability of these environments will be dependent on the project phases, and they will be available prior to the commencement of the testing phases. These test environments must be available for use throughout the duration of testing. There will be required approvals for configuration changes, as well as system coordination, which will be provided by system owners to ensure the test data remains stable, accurate, and consistent throughout the testing processes.

7.5.2. Access to other applications

To perform testing, you will need access to the Spotify API, backend, login database, and other environments. Furthermore, access to development and staging environments is required to test features, data, and environments.

7.5.3 Testing Requirements

You can commence testing once you've secured all the necessary environments, tools, and credentials. The testing team should have access to:

- A desktop browser and a mobile phone or emulator to run the Red Spotify application.
- Environment access to dev and stage.
- Access to backend systems and services.
- A developer Spotify account and OAuth credentials.
- Access to users and datasets that can be used for testing.
- Access to database (if applicable) and a query tool to validate data.
- A tool (e.g., Postman) to test API and backend service endpoints.
- Access to source code management (e.g., GitHub) and other versioning tools to view the builds, changes, and updates.
- Access to bug tracking system (e.g., Jira, Excel spreadsheet) to create, assign and track bugs.

7.5.4. Data Requirements

Data used for testing will need to be prepared ahead of time with user test accounts, playlist samples, listening history, and analytics. This includes preparing valid and invalid datasets. Spotify API test data must be created as well as tokens to ensure the platform behaves how it does in the real world.

7.5.5. Resources & Skills

This means that we require the support of a Java/Spring Boot developer, and a TypeScript/Angular developer; QA engineer to carry out testing; an API expert for Spotify integration and a team member who is familiar with databases and network troubleshooting to identify, investigate and resolve system bugs and errors.

7.6. Testing Tools

Process	Tool
Test case creation	Microsoft Word
Test case tracking	Microsoft Excel
Test case execution	Manual
Test case management	Microsoft Excel
Defect management	Microsoft Excel

7.7. Testing Handover Procedure

To hand over the results of unit testing, one must make sure that all of the tests that were written are complete. Test data needs to be documented, and all reported defects must be resolved. A review of all the test cases must be performed as well. The acceptance criteria for the testing must be satisfied.

Testing metrics would include:

- defects found*
- defects resolved*
- percentage pass of tests*
- coverage of tests*
- test measures of the performance of the system*

7.8. Testing Metrics

The metrics we'll gather are:

- number of defects by module*
- percentage of tests passed or failed*
- performance metrics for response time*
- the number of times an error is reported*

These metrics will be used to evaluate the system's robustness as well as the trouble spots in the code. We will gather the data via our defect logging system, test reports, and logs, as the results will provide us with feedback that will help with debugging, redesign, and planning of future features.

8. Test Environment Plan

8.1. Test Environment Man

Develop a Test Environment diagram.

8.2. Test Environment Details

Testers

In terms of testers, we have three to five of them. Regarding system access, we have development and staging systems, Spotify credentials, test accounts, and defect management system. In terms of hardware resources, we have laptops or desktops, mobile phones or mobile emulation, reliable internet connection, and test automation tools.

Hardware and Firmware

Laptops for the developers, mobile testing equipment and tagged servers (for the back-end and front-end) will be needed. The hardware and network equipment is used to verify the network, and also runs the app and APIs during testing. It will be provided by the team prior to testing and be used throughout testing.

Software

The software components are the Red Spotify app (latest build), Android/iOS operating system versions, Node.js, Angular, Spring Boot, and database servers. Postman, Git, and testing frameworks are the tools used. The required software is installed before starting the test and will be used throughout.

Interfaces

We utilized two APIs for the interfaces: the Spotify API and a service for authentication and data retrieval which was developed in-house. These interfaces were designed according to specifications agreed upon by our team. API endpoints were configured in development with OAuth tokens used to grant authorization to access Spotify's data; the interfaces were setup before beginning the testing stage.

Other Materials

8.3. Establishing Environment

Define the plan for establishing the testing environment, and responsibilities. This should include acquisition of each element, setup, installation and testing the environment.

Task	Requirements	Responsibility	Start Date	End Date
Environment Setup	Development and staging environments, servers, network access	Developer/Tester	Week 1	Week 2

Software Installation	Red Spotify app, APIs, testing tools	Developer/Tester	Week 2	Week 3
Environment Validation	Test data, connectivity checks, system verification	Developer/Tester	Week 3	Week 4

8.4. Environment Control

Control processes comprise of controlling versions released in the software, limiting environment access through authentication, and performing continuous monitoring of the system, while support processes enable fast resolution of any problems which may occur during this, ensuring stability and security in the whole testing phase.

8.5. Environment Roles and Responsibilities

Define the roles and responsibilities of persons who will be responsible for, or interface with the environment

Role	Staff Member	Responsibilities
Release Manager	Rowan Crooks-Taylor	Coordinates deployments and controls releases. Makes sure that environment changes are smooth and stable.
Test Manager	Isaiah Langford	Leads all tests, creates test strategies, defines scope, and manages the schedule, as well as manages the testers' work, analyzes and shares results, reports defects, and ensures testing meets quality standards, and is ready for release.
Project Manager	Davon Appolon	Plan, organize and follow the project's progress and risks to guarantee its delivery.
Document Manager	Andrew Henshaw	Ensures all documents are clear, consistent, properly formatted, and up to date.

9. Assumptions and Dependencies

9.1. Assumptions

Development support is in place to resolve identified defects as needed. We also assume that we will have consistent access to the Spotify API, that test data is ready to go, and that all development environments are set up and ready for testing. Finally, we assume testing tools, user documentation, and user permissions are all available, and that any external services will not undergo major outages during the test phase.

9.2. Dependencies

To ensure effective testing, it's critical to have development and staging environments configured, access to the required Spotify API, and the right permissions for test users. Test data must also be in place before testing can commence. In addition to this, there must be systems availability, stable builds need to be deployed prior to testing and environment preparation needs to be done.

10. Entry and Exit Criteria

The entry criteria are:

- *development completed*
- *builds deployed*
- *environments configured*
- *test data ready*

The exit criteria are the test is run successfully, critical defects are fixed, the system performs well and the application works as expected or the test is complete.

11. Administrative Plan

11.1. Approvals

Task	Responsible Person	Escalation/ Approver
Systems Integration Signoff	Rowan Crooks-Taylor	Isaiah Langford
User Acceptance Testing Signoff	Davon Appolon	Isaiah Langford
Production Verification Testing Signoff	Andrew Henshaw	Isaiah Langford

11.2. Test Milestones and Schedule

Milestone	Planned End Date	Actual End Date	Resource
Unit Testing Completion	04/19	TBD	Developer
Integration Testing Completion	04/26	TBD	Tester
System & Performance Testing Completion	05/02	TBD	Tester
Final Testing & Sign-Off	05/09	TBD	Developer/Test

11.3. Training

Training Requirement	Staff	Date
Application Overview	Testers	04/19
Spotify API and OAuth Usage	Testers/Developers	04/19
Testing Tools Training	Testers	04/26
Test Case Execution and Defect Logging	Testers	04/26

11.4. Defect Management

Microsoft Word will be used for defect logging, tracking and prioritizing. Severity levels will be set for all logged issues and assigned to an engineer for resolution. All defects will be subject to a defect lifecycle and will move through the appropriate defect statuses.

12. Definitions

Term/Acronym	Definition
API (Application Programming Interface)	Application programming interfaces (APIs) are a collection of protocols and standards that enable interactions between software applications.
OAuth 2.0	OAuth 2.0 is an authorization standard developed and utilized across the industry to allow an application to access your data without asking for your passwords to do so.
Access Token	Once you successfully complete the login process, the application issues a temporary credential known as an access token to authenticate further authorized API requests on behalf of the user.
Refresh Token	If your current access token expires, refresh tokens provide you a new access token. In this way, refresh tokens allow you to keep your user sessions active and keep refreshing them as often as necessary.

13. References

The following documents have been used to assist in creation of this document.

#	Document name	Version	Comments
1	Spotify for Developers	Latest	
2	OAuth 2.0 Authorization Framework	RFC 6749	
3	ISO/IEC/IEEE 29119 Software Testing Standard	Latest	